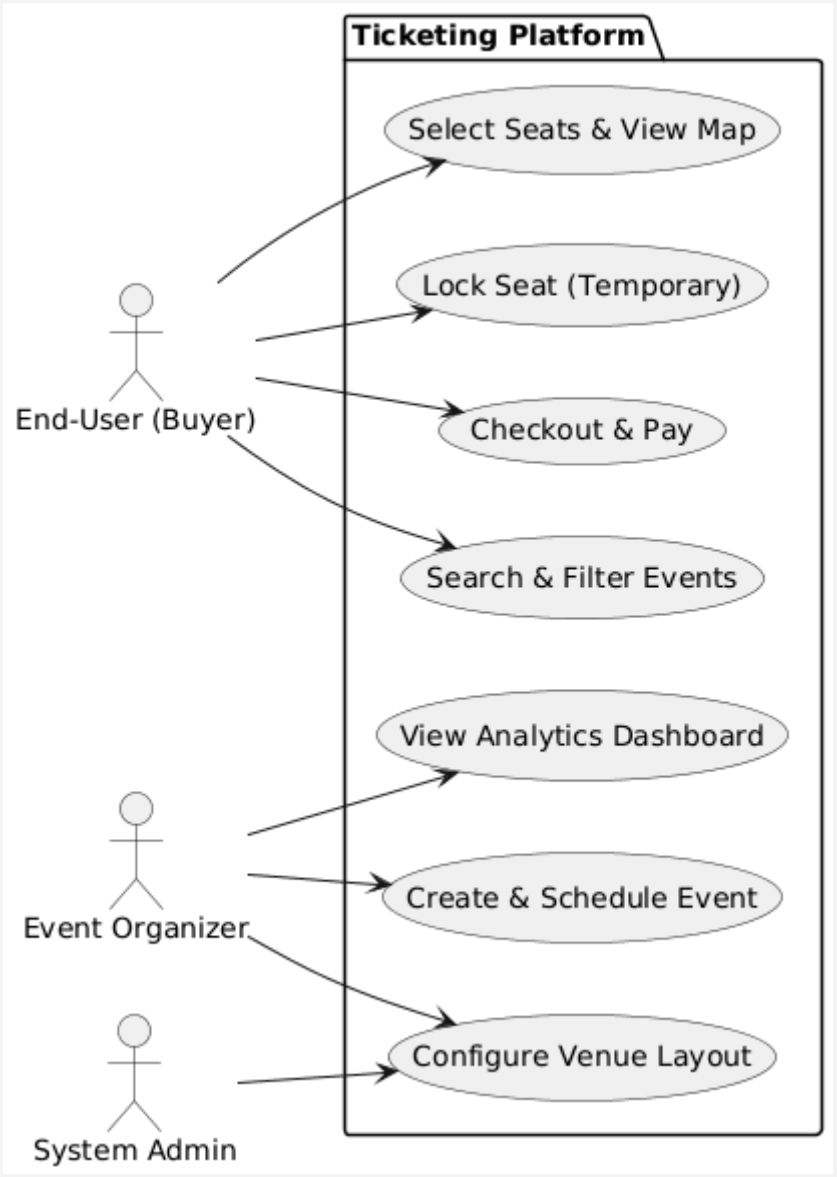
UML Architecture Models: Event Ticketing Platform

Project: Software Engineering II
Team Members: Behrad Hozouri 40331411
Amir Kalaki 40131038

1. Use Case Diagram

Illustrates the primary actors and their interactions with the ticketing system.

```
@startuml
left to right direction
actor "End-User (Buyer)" as user
actor "Event Organizer" as organizer
actor "System Admin" as admin

package "Ticketing Platform" {
  usecase "Search & Filter Events" as UC1
  usecase "Select Seats & View Map" as UC2
  usecase "Lock Seat (Temporary)" as UC3
  usecase "Checkout & Pay" as UC4

  usecase "Configure Venue Layout" as UC5
  usecase "Create & Schedule Event" as UC6
  usecase "View Analytics Dashboard" as UC7
}

user --> UC1
user --> UC2
user --> UC3
user --> UC4

organizer --> UC5
organizer --> UC6
organizer --> UC7

admin --> UC5
@enduml
```

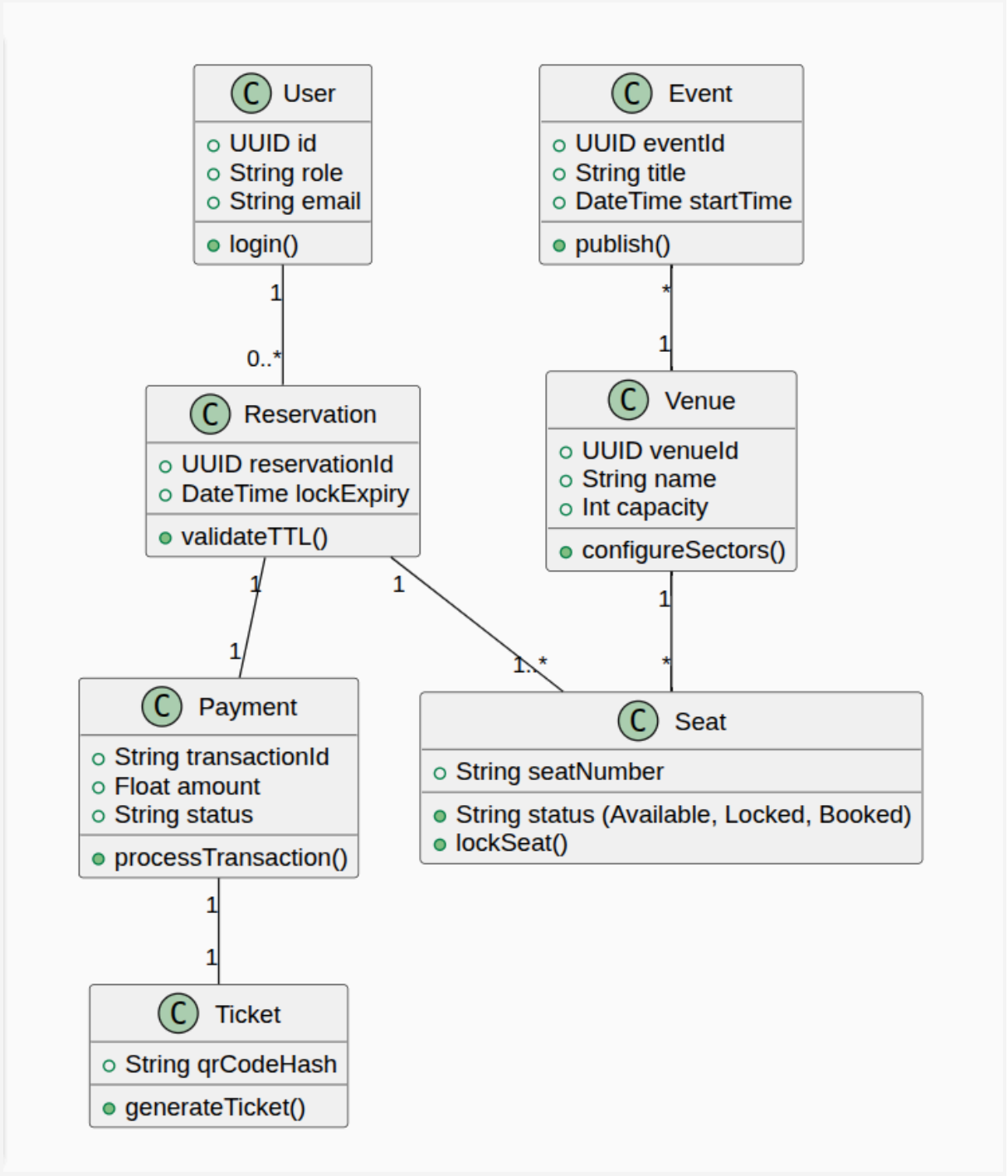


2. Class Diagram

Maps the core data entities and their underlying business relationships.

```
@startuml
class User {
  +UUID id
  +String role
  +String email
  +login()
}

class Event {
  +UUID eventId
```

```
  +String title
  +DateTime startTime
  +publish()
}

class Venue {
  +UUID venueId
  +String name
  +Int capacity
  +configureSectors()
}

class Seat {
  +String seatNumber
  +String status (Available, Locked, Booked)
  +lockSeat()
}

class Reservation {
  +UUID reservationId
  +DateTime lockExpiry
  +validateTTL()
}

class Payment {
  +String transactionId
  +Float amount
  +String status
  +processTransaction()
}

class Ticket {
  +String qrCodeHash
  +generateTicket()
}

User "1" -- "0..*" Reservation
Event "*" -- "1" Venue
Venue "1" -- "*" Seat
Reservation "1" -- "1..*" Seat
Reservation "1" -- "1" Payment
Payment "1" -- "1" Ticket
@enduml
```

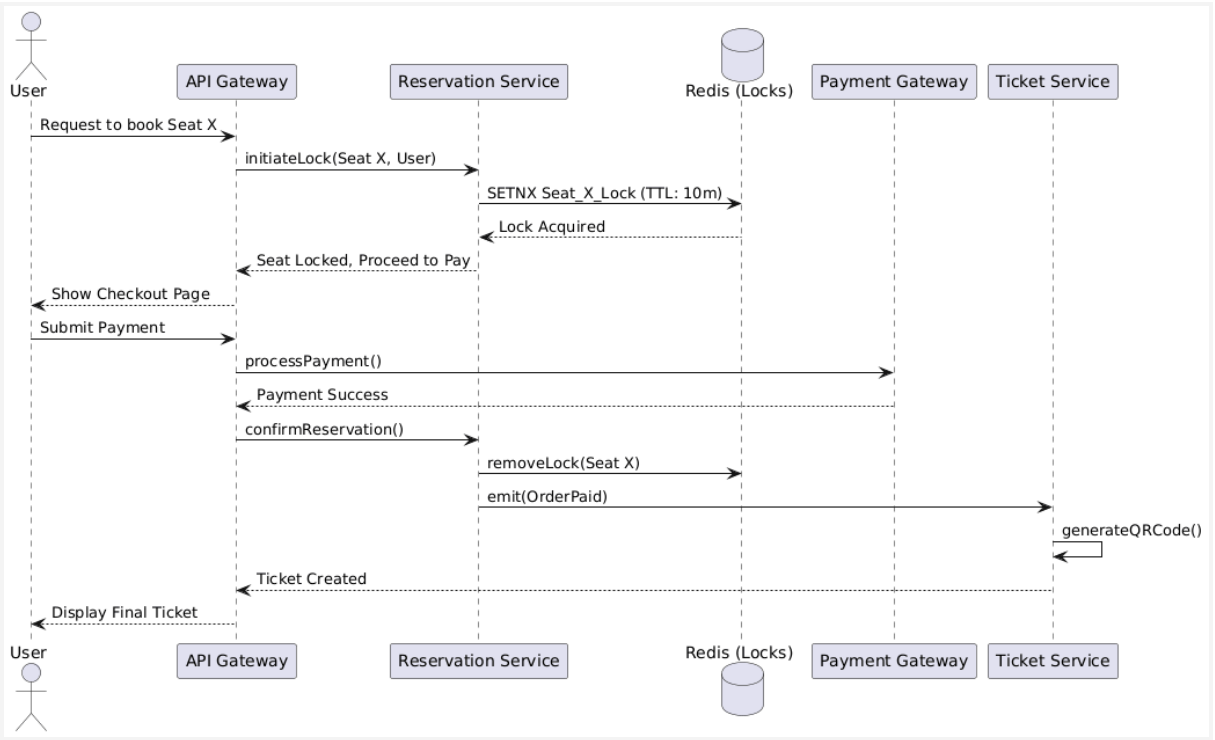


3. Sequence Diagram (End-to-End Booking Flow)

Visualizes the step-by-step ticket purchase, including Redis locking and payment.

```
@startuml
actor User
participant "API Gateway" as API
participant "Reservation Service" as RS
```

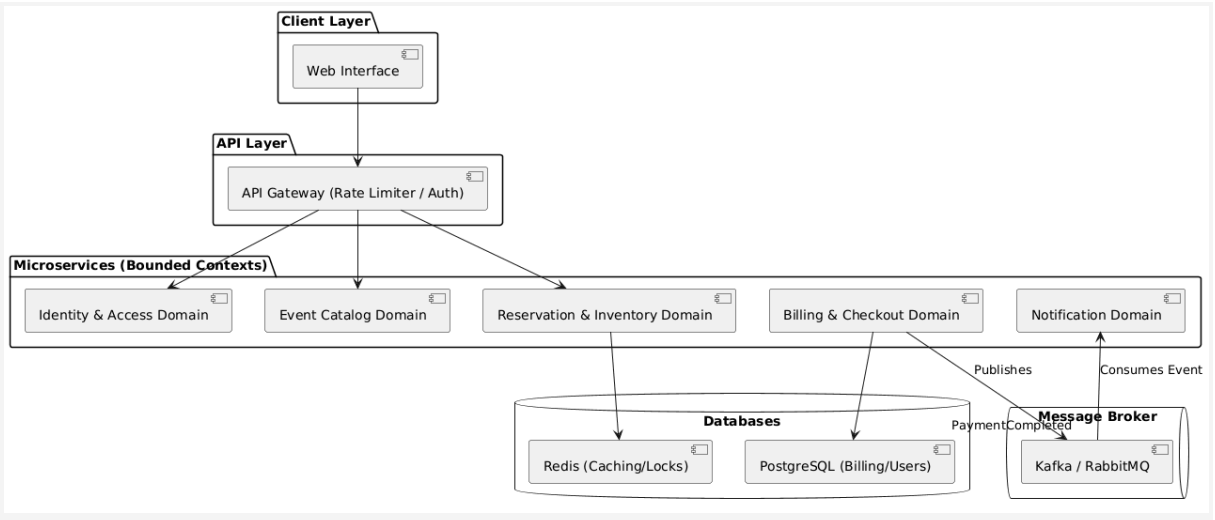
```
database "Redis (Locks)" as Redis
participant "Payment Gateway" as PG
participant "Ticket Service" as TS

User -> API: Request to book Seat X
API -> RS: initiateLock(Seat X, User)
RS -> Redis: SETNX Seat_X_Lock (TTL: 10m)
Redis --> RS: Lock Acquired
RS --> API: Seat Locked, Proceed to Pay
API --> User: Show Checkout Page

User -> API: Submit Payment
API -> PG: processPayment()
PG --> API: Payment Success
API -> RS: confirmReservation()
RS -> Redis: removeLock(Seat X)
RS -> TS: emit(OrderPaid)
TS -> TS: generateQRCode()
TS --> API: Ticket Created
API --> User: Display Final Ticket
@enduml
```



4. Component Diagram

Illustrates the high-level functional boundaries and Domain-Driven Design (DDD) contexts.

```
@startuml
package "Client Layer" {
  [Web Interface]
}

package "API Layer" {
  [API Gateway (Rate Limiter / Auth)]
}

package "Microservices (Bounded Contexts)" {
  [Identity & Access Domain]
  [Event Catalog Domain]
  [Reservation & Inventory Domain]
  [Billing & Checkout Domain]
  [Notification Domain]
}

database "Databases" {
  [PostgreSQL (Billing/Users)]
  [Redis (Caching/Locks)]
}

queue "Message Broker" {
  [Kafka / RabbitMQ]
}

[Web Interface] --> [API Gateway (Rate Limiter / Auth)]
[API Gateway (Rate Limiter / Auth)] --> [Identity & Access Domain]
[API Gateway (Rate Limiter / Auth)] --> [Event Catalog Domain]
[API Gateway (Rate Limiter / Auth)] --> [Reservation & Inventory Domain]

[Reservation & Inventory Domain] --> [Redis (Caching/Locks)]
[Billing & Checkout Domain] --> [PostgreSQL (Billing/Users)]

[Billing & Checkout Domain] --> [Kafka / RabbitMQ] : Publishes
"PaymentCompleted"
[Kafka / RabbitMQ] --> [Notification Domain] : Consumes Event
@enduml
```



5. Deployment Diagram

Maps the container orchestration across Kubernetes clusters.

```
@startuml
node "Client Device" {
  [Browser / Mobile App]
}

node "Cloud Environment (Kubernetes Cluster)" {
  node "Ingress Controller" {
    [Load Balancer]
  }

  node "Go Microservices Pods" {
    [Reservation Engine]
    [Billing Service]
    [Catalog Service]
  }

  node "Stateful Data Nodes" {
    database "PostgreSQL Primary"
    database "Redis Cluster"
    queue "Kafka Brokers"
  }
}

[Browser / Mobile App] --> [Load Balancer] : HTTPS
[Load Balancer] --> [Reservation Engine]
[Load Balancer] --> [Billing Service]
[Reservation Engine] --> [Redis Cluster] : Transient Locks
```

```
[Billing Service] --> [PostgreSQL Primary] : ACID Transactions
[Billing Service] --> [Kafka Brokers] : Async Events
@enduml
```